Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Авдей Д.Б.
Проверил:
Кулик А.Д.

Брест 2026

Цель: приобрести практические навыки разработки API и баз данных.

Вариант №1

Задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

1) База данных Деканат

Результат:

Code Details

200

Response body


```
[
  {
    "id": 3,
    "name": "ФЭИС*",
    "dean_name": "Разумейчик В.С.*"
  },
  {
    "id": 2,
    "name": "МСФ",
    "dean_name": "Онысько С.Р."
  }
]
```


Download

Response headers

```
content-length: 133
content-type: application/json
date: Wed,18 Mar 2026 16:47:53 GMT
server: uvicorn
```

Responses

```
INFO:     Will watch for changes in these directories: ['E:\\program\\spp_po12\\reports\\Avdey\\Lab_5\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [13424] using StatReload
INFO:     Started server process [9188]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Name Description

faculty_id * required
integer
(path)

3

Code Details

200

Response body

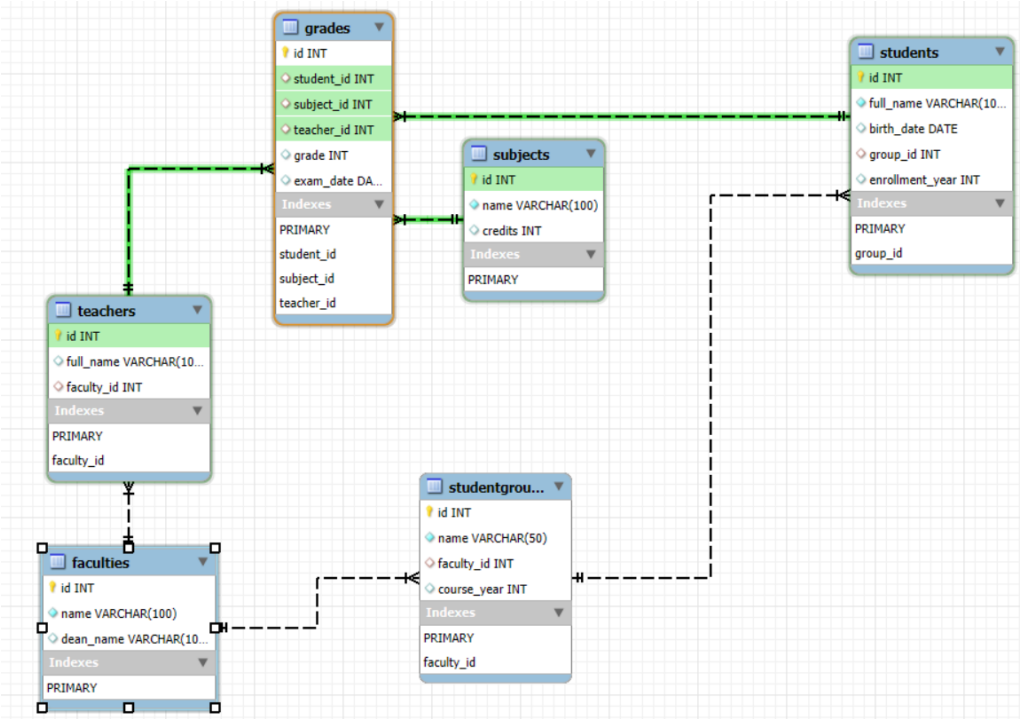

```
{
  "message": "Faculty deleted"
}
```


Download

Response headers

default

POST	/faculties/	Create Faculty
GET	/faculties/	Get Faculties
PUT	/faculties/{faculty_id}	Update Faculty
DELETE	/faculties/{faculty_id}	Delete Faculty
POST	/groups/	Create Group
GET	/groups/	Get Groups
PUT	/groups/{group_id}	Update Group
DELETE	/groups/{group_id}	Delete Group
POST	/students/	Create Student
GET	/students/	Get Students
PUT	/students/{student_id}	Update Student
DELETE	/students/{student_id}	Delete Student
POST	/subjects/	Create Subject
GET	/subjects/	Get Subjects
PUT	/subjects/{subject_id}	Update Subject
DELETE	/subjects/{subject_id}	Delete Subject
POST	/teachers/	Create Teacher
GET	/teachers/	Get Teachers
PUT	/teachers/{teacher_id}	Update Teacher
DELETE	/teachers/{teacher_id}	Delete Teacher
POST	/grades/	Create Grade
GET	/grades/	Get Grades
PUT	/grades/{grade_id}	Update Grade
DELETE	/grades/{grade_id}	Delete Grade



Код:

```
from fastapi import FastAPI, HTTPException
from sqlalchemy import create_engine, Column, Integer, String, Date,
ForeignKey
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "mysql+pymysql://root:root@localhost/deanery"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

app = FastAPI(title="Deanery API")


class Faculty(Base):
    __tablename__ = "Faculties"
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    dean_name = Column(String(100))


class StudentGroup(Base):
    __tablename__ = "StudentGroups"
    id = Column(Integer, primary_key=True)
    name = Column(String(50), nullable=False)
    faculty_id = Column(Integer, ForeignKey("Faculties.id"))
    course_year = Column(Integer)


class Student(Base):
    __tablename__ = "Students"
    id = Column(Integer, primary_key=True)
    full_name = Column(String(100), nullable=False)
    birth_date = Column(Date)
    group_id = Column(Integer, ForeignKey("StudentGroups.id"))
    enrollment_year = Column(Integer)


class Subject(Base):
    __tablename__ = "Subjects"
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    credits = Column(Integer)


class Teacher(Base):
    __tablename__ = "Teachers"
    id = Column(Integer, primary_key=True)
    full_name = Column(String(100), nullable=False)
    faculty_id = Column(Integer, ForeignKey("Faculties.id"))


class Grade(Base):
    __tablename__ = "Grades"
    id = Column(Integer, primary_key=True)
    student_id = Column(Integer, ForeignKey("Students.id"))
    subject_id = Column(Integer, ForeignKey("Subjects.id"))
    teacher_id = Column(Integer, ForeignKey("Teachers.id"))
    grade = Column(Integer)
    exam_date = Column(Date)


Base.metadata.create_all(engine)
```

```
@app.post("/faculties/")
def create_faculty(name: str, dean_name: str):
    db = SessionLocal()
    f = Faculty(name=name, dean_name=dean_name)
    db.add(f)
    db.commit()
    return {"message": "Faculty added"}


@app.get("/faculties/")
def get_faculties():
    db = SessionLocal()
    return db.query(Faculty).all()


@app.put("/faculties/{faculty_id}")
def update_faculty(faculty_id: int, name: str = None, dean_name: str = None):
    db = SessionLocal()
    f = db.query(Faculty).get(faculty_id)
    if not f:
        raise HTTPException(status_code=404, detail="Faculty not found")
    if name:
        f.name = name
    if dean_name:
        f.dean_name = dean_name
    db.commit()
    return {"message": "Faculty updated"}


@app.delete("/faculties/{faculty_id}")
def delete_faculty(faculty_id: int):
    db = SessionLocal()
    f = db.query(Faculty).get(faculty_id)
    if not f:
        raise HTTPException(status_code=404, detail="Faculty not found")
    db.delete(f)
    db.commit()
    return {"message": "Faculty deleted"}


@app.post("/groups/")
def create_group(name: str, faculty_id: int, course_year: int):
    db = SessionLocal()
    g = StudentGroup(name=name, faculty_id=faculty_id,
course_year=course_year)
    db.add(g)
    db.commit()
    return {"message": "Group added"}


@app.get("/groups/")
def get_groups():
    db = SessionLocal()
    return db.query(StudentGroup).all()


@app.put("/groups/{group_id}")
def update_group(group_id: int, name: str = None, faculty_id: int = None,
course_year: int = None):
    db = SessionLocal()
    g = db.query(StudentGroup).get(group_id)
    if not g:
```

```
        raise HTTPException(status_code=404, detail="Group not found")
    if name:
        g.name = name
    if faculty_id:
        g.faculty_id = faculty_id
    if course_year:
        g.course_year = course_year
    db.commit()
    return {"message": "Group updated"}


@app.delete("/groups/{group_id}")
def delete_group(group_id: int):
    db = SessionLocal()
    g = db.query(StudentGroup).get(group_id)
    if not g:
        raise HTTPException(status_code=404, detail="Group not found")
    db.delete(g)
    db.commit()
    return {"message": "Group deleted"}


@app.post("/students/")
def create_student(name: str, group_id: int, enrollment_year: int,
birth_date: str = None):
    db = SessionLocal()
    s = Student(full_name=name, group_id=group_id,
enrollment_year=enrollment_year, birth_date=birth_date)
    db.add(s)
    db.commit()
    return {"message": "Student added"}


@app.get("/students/")
def get_students():
    db = SessionLocal()
    return db.query(Student).all()


@app.put("/students/{student_id}")
def update_student(student_id: int, name: str = None, group_id: int = None,
enrollment_year: int = None):
    db = SessionLocal()
    s = db.query(Student).get(student_id)
    if not s:
        raise HTTPException(status_code=404, detail="Student not found")
    if name:
        s.full_name = name
    if group_id:
        s.group_id = group_id
    if enrollment_year:
        s.enrollment_year = enrollment_year
    db.commit()
    return {"message": "Student updated"}


@app.delete("/students/{student_id}")
def delete_student(student_id: int):
    db = SessionLocal()
    s = db.query(Student).get(student_id)
    if not s:
        raise HTTPException(status_code=404, detail="Student not found")
    db.delete(s)
    db.commit()
```

```
    return {"message": "Student deleted"}


@app.post("/subjects/")
def create_subject(name: str, credits_value: int):
    db = SessionLocal()
    sub = Subject(name=name, credits=credits_value)
    db.add(sub)
    db.commit()
    return {"message": "Subject added"}


@app.get("/subjects/")
def get_subjects():
    db = SessionLocal()
    return db.query(Subject).all()


@app.put("/subjects/{subject_id}")
def update_subject(subject_id: int, name: str = None, credits_value: int =
None):
    db = SessionLocal()
    sub = db.query(Subject).get(subject_id)
    if not sub:
        raise HTTPException(status_code=404, detail="Subject not found")
    if name:
        sub.name = name
    if credits:
        sub.credits = credits_value
    db.commit()
    return {"message": "Subject updated"}


@app.delete("/subjects/{subject_id}")
def delete_subject(subject_id: int):
    db = SessionLocal()
    sub = db.query(Subject).get(subject_id)
    if not sub:
        raise HTTPException(status_code=404, detail="Subject not found")
    db.delete(sub)
    db.commit()
    return {"message": "Subject deleted"}


@app.post("/teachers/")
def create_teacher(name: str, faculty_id: int):
    db = SessionLocal()
    t = Teacher(full_name=name, faculty_id=faculty_id)
    db.add(t)
    db.commit()
    return {"message": "Teacher added"}


@app.get("/teachers/")
def get_teachers():
    db = SessionLocal()
    return db.query(Teacher).all()


@app.put("/teachers/{teacher_id}")
def update_teacher(teacher_id: int, name: str = None, faculty_id: int =
None):
    db = SessionLocal()
    t = db.query(Teacher).get(teacher_id)
```

```
    if not t:
        raise HTTPException(status_code=404, detail="Teacher not found")
    if name:
        t.full_name = name
    if faculty_id:
        t.faculty_id = faculty_id
    db.commit()
    return {"message": "Teacher updated"}


@app.delete("/teachers/{teacher_id}")
def delete_teacher(teacher_id: int):
    db = SessionLocal()
    t = db.query(Teacher).get(teacher_id)
    if not t:
        raise HTTPException(status_code=404, detail="Teacher not found")
    db.delete(t)
    db.commit()
    return {"message": "Teacher deleted"}


@app.post("/grades/")
def create_grade(student_id: int, subject_id: int, teacher_id: int, grade:
int, exam_date: str = None):
    db = SessionLocal()
    g = Grade(student_id=student_id, subject_id=subject_id,
teacher_id=teacher_id, grade=grade, exam_date=exam_date)
    db.add(g)
    db.commit()
    return {"message": "Grade added"}


@app.get("/grades/")
def get_grades():
    db = SessionLocal()
    return db.query(Grade).all()


@app.put("/grades/{grade_id}")
def update_grade(
    grade_id: int, student_id: int = None, subject_id: int = None,
teacher_id: int = None, grade: int = None
):
    db = SessionLocal()
    g = db.query(Grade).get(grade_id)
    if not g:
        raise HTTPException(status_code=404, detail="Grade not found")
    if student_id:
        g.student_id = student_id
    if subject_id:
        g.subject_id = subject_id
    if teacher_id:
        g.teacher_id = teacher_id
    if grade is not None:
        g.grade = grade
    db.commit()
    return {"message": "Grade updated"}


@app.delete("/grades/{grade_id}")
def delete_grade(grade_id: int):
    db = SessionLocal()
    g = db.query(Grade).get(grade_id)
    if not g:
```

```
        raise HTTPException(status_code=404, detail="Grade not found")
    db.delete(g)
    db.commit()
    return {"message": "Grade deleted"}


# uvicorn lab_5:app --reload
```

Вывод: приобрел практические навыки разработки API и баз данных.